

Installare CDuce

Davide Camino

February 15, 2025

Abstract

Vediamo come installare alcune versioni di CDuce, sfruttando gli *switch* di opam, ovvero degli ambienti isolati di OCaml.

1 Introduzione

Attraverso gli *switch* opam consente di gestire differenti installazioni che possono coesistere le une con le altre. La filosofia è simile agli ambienti virtuali di Python (*virtualenv*). Ogni *switch* è indipendente dagli altri, può avere i suoi pacchetti e le sue configurazioni. La cosa più interessante ai nostri fini è la possibilità di ogni *switch* di avere la propria versione di OCaml con le sue librerie e i suoi binari.

Quello che vogliamo fare quindi è avere uno *switch* per ogni installazione di CDuce, senza doverci preoccupare di gestire le compatibilità perché andremo ad usare in ogni *switch* la versione di OCaml che ci serve per compilare la versione di CDuce che vogliamo installare.

Visualizzare gli *switch*

Una volta installato opam verrà creato uno *switch* di default possiamo visualizzare tutti gli *switch* presenti con:

```
1 $ opam switch list
2 #   switch   compiler   description
3 -> default  ocaml.4.13.1  default
```

Creare un nuovo *switch*

Per creare un nuovo *switch* è sufficiente:

```
1 $ opam switch create my_project <compiler-version>
```

Adesso entriamo nel nuovo *switch* con:

```
1 $ opam switch my_project
```

Eliminare uno *switch*

Per eliminare dal disco uno *switch* non più necessario è sufficiente eseguire:

```
1 $ opam switch remove my_project
```

2 Installare CDuce

2.1 Versione Stable Polymorphic

Questa è la versione il cui codice si trova attualmente nel branch `main`, gli ultimi aggiornamenti risalgono ad un paio di anni fa e teoricamente le istruzioni per l'installazione sono quelle presenti sul sito (www.cduce.org).

Purtroppo da allora il sito e il software non sono stati aggiornati, mentre le dipendenze sì, ropendo la compatibilità e rendendo impossibile seguire quelle istruzioni.

Per cercare di ripristinare corrispondenza tra versione di CDuce e di OCaml ho cercato quale fosse la data dell'ultimo commit sul branch `main` e ho scaricato la versione di OCaml precedente alla data dell'ultimo commit, sperando in questo modo di ripristinare la compatibilità tra i linguaggi. L'idea era giusta e sono riuscito a installare CDuce come pacchetto di OCaml in uno *switch* apposito.

È particolarmente importante installare CDuce come pacchetto di OCaml e non semplicemente compilarlo per avere un eseguibile perché (come ci ha spiegato Kim) nel primo modo si ha una interazione molto più forte tra i due linguaggi potendo richiamare l'uno dall'altro e viceversa.

Vediamo passo passo come installare questa versione di CDuce

Creare e configurare *switch*

Creiamo un nuovo *switch* che ospiterà come pacchetto CDuce

```
1 $ opam switch create CDuce_poly_stable 4.13.1
```

Il nome `CDuce_poly_stable` è assolutamente arbitrario, mentre la versione 4.13.1 risale al 01/10/2021, cioè è l'ultima versione precedente all'ultimo commit sul branch `main`. Adesso dovremmo trovarci nello *switch* appena creato, possiamo verificarlo con `$ opam switch` e nel caso non ci trovassimo nello *switch* giusto possiamo andarci con `opam switch CDuce_poly_stable`.

Installiamo ora tutti i pacchetti necessari:

```
1 $ opam install conf-libssl dune js_of_ocaml-compiler \  
2   js_of_ocaml-ppx markup menhir menhirLib ocaml \  
3   ocaml-compiler-libs ocaml-expat ocamlfind ocamlnet \  
4   ocurl odoc ppx sedlex zarith zarith_stubs_js
```

Opam si preoccuperà di installare tutte le dipendenze necessarie, se le dipendenze vanno installate col package manager della tua distro possono essere necessari i privilegi di amministratore (`sudo`) che chiederà direttamente opam.

Installare CDuce

Come suggerito da Kim installiamo CDuce facendo il *clone* del repository

```
1 $ git clone https://gitlab.math.univ-paris-diderot.fr/cduce/cduce.git
2 $ cd cduce
```

Prima di procedere esploriamo il contenuto della cartella ad esempio con il comando `ls`. Possiamo notare 3 file con estensione `opam` (`cduce-tools.opam`, `cduce-types.opam` e `cduce.opam`) questi saranno i pacchetti che chiederemo ad `opam` di installare.

Siamo pronti per installare CDuce, per farlo è sufficiente:

```
1 $ opam pin .
```

`Opam` riconoscerà i pacchetti presenti nella cartella (con estensione `opam`) e cercherà di installarli. Durante l'installazione `opam` chiederà conferma prima di procedere, rispondendo di sì a tutte le domande e dopo aver atteso il tempo di compilazione avremo uno *switch* con CDuce installato.

Per verificare che tutto sia andato a buon fine possiamo ora eseguire:

```
1 $ cduce --version
2 CDuce, version poly-devel-97-g3cba0c5
3 Using OCaml 4.13.1 compiler
4 Supported features:
5 - netclient: Load external URLs with netclient
6 - pxp: PXP XML parser
7 - netstring: Load HTML document with netstring
8 - system: System calls
9 - ocaml: OCaml interface
```

È importante notare che CDuce è adesso stato installato come pacchetto in uno *switch*, sarà `opam` che si occuperà di cercare l'eseguibile per noi, quindi possiamo invocare il comando `cduce` in qualsiasi punto del filesystem.

2.2 Versione attuale Dev

Questa è la versione che si trova sul branch `dev` ed è quella sulla quale sta attualmente lavorando Kim. Di questa sono già presenti le istruzioni, le riporto qui in forma stringata solo per completezza.

Creare e configurare *switch*

Creiamo lo *switch* con la versione corretta del compilatore

```
1 $ opam switch create CDuce_poly_dev 4.14.2
```

Ci spostiamo nello *switch* (se necessario) e installiamo tutti i pacchetti che servono per compilare CDuce.

```
1 $ opam switch CDuce_poly_dev
2 $ opam install conf-libssl dune js_of_ocaml-compiler \
3 js_of_ocaml-ppx markup menhir menhirLib ocaml \
4 ocaml-compiler-libs ocaml-expat ocamlfind ocamlnet \
5 ocurl odoc ppx sedlex zarith zarith_stubs_js
```

Installare CDuce

In una nuova posizione del filesystem rispetto alla cartella precedente facciamo il *clone* del repository e ci spostiamo sul branch *dev*

```
1 $ git clone https://gitlab.math.univ-paris-diderot.fr/cduce/cduce.git
2 $ cd cduce
3 $ git checkout dev
```

Installiamo CDuce con

```
1 $ opam pin .
```

Nuovamente sarà necessario rispondere di sì a tutte le domande e attendere la compilazione del pacchetto. Una volta terminato il processo possiamo nuovamente verificare l'installazione con

```
1 $ cduce --version
2 CDuce, version poly-devel-319-g3e36d3f
3 Using OCaml 4.14.2 compiler
4 Supported features:
5 - netclient: Load external URLs with netclient
6 - ppx: PXP XML parser
7 - netstring: Load HTML document with netstring
8 - system: System calls
9 - curl: Load external URLs with curl
10 - expat: Expat XML parser
11 - ocaml: OCaml interface
12 - markup: Markup.ml XML and HTML parser
```

2.3 Passare da una versione all'altra

Possiamo sfruttare gli *switch* per passare comodamente da una versione all'altra di CDuce. Ogni versione infatti è legata al suo *switch* e quando usiamo il comando `cduce` viene selezionato l'eseguibile installato nello *switch* in cui siamo. È sufficiente spostarsi nello *switch* in cui si è installata la versione di CDuce che si desidera utilizzare (`opam switch <nome_switch>`) e invocare il comando `cduce` .

3 Esperimenti

Sulla versione che qui è chiamata **attuale dev** non ho fatto ulteriori esperimenti. Ho però fatto dei test sulla versione **stable**.

3.1 Test base

Ho provato a ripetere l'esperimento che avevamo già fatto negli scorsi giorni: caricare una semplicissima ontologia e farne il parsing. Il codice e l'ontologia si trovano qui: github.com/DavideCamino/experiment_on_CDuce.

Questo test è andato a buon fine. Sono allora passato a verificare funzionalità più avanzate.

3.2 Da triple ad ontologia

In questo set di test ho provato a:

- caricare un file XML
- fare il parsing di un documento XML
- definire la struttura di un'ontologia
- trasformare il documento XML in un'ontologia
- esportare l'ontologia appena creata in modo da poterla visualizzare con strumenti appositi (protegè)

Le prove fatte ripercorrono gli esperimenti che avevamo fatto per presentare CDuce ad una conferenza sui linguaggi funzionali, tutto il codice e una breve relazione del lavoro si trovano qui: github.com/DavideCamino/Triple.to.Ontology.

Tutti i test hanno avuto esito positivo, il caricamento non ha presentato strane ripetizioni dei namespaces, la definizione della struttura dell'ontologia e il parsing sono avvenuti correttamente, infine anche il dump su file è stato fatto con successo (ancora come già succedeva nelle altre versioni tutto l'output si trova su una sola riga, ma non è un grande problema).

3.3 Integrazione con OCaml

Seguendo quanto riportato sulla guida di Kim (github.com/Tchou/cduce-sample) ho provato a far interagire OCaml con CDuce.

Ho provato a seguire fedelmente l'esempio che ha riportato Kim nel repository indicato sopra e tutto ha funzionato correttamente. Si riescono ad usare in CDuce funzioni della libreria standard di OCaml, si possono integrare in CDuce funzioni arbitrarie scritte in OCaml e si possono richiamare funzioni scritte in CDuce in OCaml. È infine possibile compilare il codice sorgente per ottenere un eseguibile che può essere lanciato senza bisogno di `cduce --run <nome-eseguibile>.cdo`.

È importante sottolineare ancora una volta che l'integrazione è possibile solo perché abbiamo installato CDuce come pacchetto all'interno di uno *switch* di OCaml, se avessimo ottenuto direttamente un eseguibile compilando i sorgenti (anche per questa procedura è possibile trovare le istruzioni su www.cduce.org) non avremmo potuto far interagire i due strumenti.

4 Lavoro futuro

Ora che abbiamo una versione relativamente stabile in cui sembra funzionare tutto potrebbe essere interessante cercare di seguire le istruzioni di Kim per creare dei makefile organizzati dei nostri sorgenti, in questo modo possiamo compilare il codice scritto in CDuce con facilità.

Inoltre potrebbe essere interessante cercare di integrare i due linguaggi (OCaml e CDuce) anche nei nostri progetti. OCaml è infatti vastissimo e ci sono già un sacco di librerie che potenzialmente potrebbero trinarci utili, avere la possibilità di sfruttare tutta la potenza di OCaml mi sembra una strada che può valer la pena esplorare.

5 Conclusioni

Siamo ora in grado di creare nuovi *switch* dove testare le nuove versioni di CDuce man mano che lo sviluppo procede.

Abbiamo individuato una versione che potrebbe fungere da baseline nel quale sembrerebbe che tutto funzioni correttamente e che possiamo usare mentre vengono sviluppate nuove funzionalità.

Siamo quindi in grado di avere un ambiente stabile nel quale testare tutte le nuove idee, dal punto di vista del setup del linguaggio questa configurazione mi sembra la migliore che possiamo avere al momento, mi sembra inoltre sufficientemente funzionale da poterci permettere di concentrarci sul **vero** lavoro di ricerca.